
Reproducibility Report - The Classification of White Wine and Red Wine According to Their Physicochemical Qualities

Sahar S.

Electrical Engineering Department
École de Technologie Supérieure University
sahar.shahd.1@ens.etsmtl.ca

Brent Y.

Department of Computer Science
McGill University
ji.s.yoon@mail.mcgill.ca

Reproducibility Summary

Scope of Reproducibility

According to the context of the paper [1], the main purpose is to investigate the wine type and quality prediction based on physicochemical data. The datasets utilized are taken from "UC Irvine Machine Learning Repository" [5]. Three different algorithms of Random Forest (RF), k-nearest-neighbourhood (KNN) and support vector machines (SVM) are investigated and the final result is also provided with Principal Component Analysis. It is claimed that the best result is achieved by using Random Forest algorithm in combination with Principal Component Analysis for feature selection.

Methodology

Due to the inaccessibility of the original code, we applied the algorithms from scikit-learn library. To investigate the experiments, the same datasets as the paper are preprocessed and used. The codes are setup in Google Colaboratory with minimum available computational resources.

Results

In our evaluations, our results could also represent that the Random Forest model is the most accurate method among these three for wine type classification and quality classification. The results also support that principle component analysis (PCA) improves the accuracy of red wine but decreases white wine accuracy. The accuracy of cross validation for type classification for SVM, KNN, and RF were 99.5%, 99.1%, 99.8% respectively, which supports that RF is the most accurate for type classification. The red wine quality classification accuracy for SVM, KNN, RF were 29.3%, 64.5%, 70.9%, respectively; with PCA RF accuracy increased to 72.6%. These results both support at RF is the best for quality classification and that PCA improves red wine accuracy while decreasing white wine accuracy. The accuracy of KNN and SVM were more difficult to reproduce compared to RF.

What was easy

As a result of comprehensive descriptions of the paper's objective, the implementation procedures were understandable and accessible. Thanks to the scikit-learn library [2], the project's re-implementation was considerably easy. Most of the objectives of the paper were obtained by the suggested configurations. It is not out of scope to mention that fine-tuning the hyper parameters are required even using the straightforward libraries.

What was difficult

The exact parameters for all the models and the finer details of the evaluation models were not provided by the paper making the reproduction of all the accuracy infeasible. The scoring metrics of the quality classifications were considerably harder to obtain than the type classification. For multi-class, we had to consider averaging techniques and handling cases where the stratified k fold did not enough label instances.

Communication with original authors

No correspondence was exchanged with the authors of the original paper.

1 Introduction

The amount of different types of wines consumed today are vast and the industry is trying to implement new technologies to aid the making and selling processes of wine [3]. Among other factors wine certifications are based off of their physicochemical and sensory test [4]. For consistency and economic reasons it is important to classify wine products. Data mining techniques can be used, the goal of these machine learning algorithms is to accurately predict the wine quality.

In the associated paper three machine learning methods are compared for their effectiveness in accurately predicting the wine qualities of red and white wines given features related to the physicochemical properties of the wines. The methods are k-nearest-neighbourhood (KNN), support vector machines (SVM), and Random Forest. These algorithms in their principle mechanisms and decision making processes, each with their own advantages and disadvantages. The objective is to identify which machine performs best for this specific task. The dataset is provided by UCI Machine Learning Repository: <https://archive.ics.uci.edu/ml/datasets/Wine+Quality> [5].

Owing to the large dataset size and the large number of features we also briefly explore the capabilities of principle component analysis (PCA) within this study. PCA is a dimensionality reduction method that transforms the large data into a smaller set that still is able to retain most of the information from the initial data.

2 Scope of reproducibility

As mentioned previously, the main objective of this paper is to evaluate the application of three different machine learning algorithms on wine quality prediction. The more exploration on the paper revealed that the type prediction has also been under investigation. In accordance with the result provided, we can see that the performance of the algorithms are represented with two forms of cross validation and percentage split. Conforming to the results, they are illustrated in separate tables with the following descriptions:

- Performance results of the classification of red and white wine samples.
- Performance results of the classification of red and white wine sample qualities
- Performance results of the classification of red and white wine sample qualities after applying PCA.

Based on the paper's context, following claims are raised which are also supported by the experiments which we conducted:

- Among the three algorithms under investigation, Random Forest is able to obtain the highest accuracy for type classification.
- Random Forest also is the most successful algorithm to classify the wine quality accurately.
- Utilizing PCA improves the performance of Random Forest for red wines quality classification. However, it results in lower accuracy for white wine quality classification.

Experiment in Section 4 will represent our results comparing with the authors claims.

3 Methodology

The author's code was unavailable thus we needed to reimplement the methods of their paper. The models and principle component analysis were implemented using the scikit-learn library. We referenced the scikit-learn methods documentations to view the parameters and the output elements in order to implement the machines and obtain a score for the algorithms. A majority of the parameters for the methods were not disclosed by the paper. So, we stayed consistent for the parameters that were mentioned and kept all other parameters as defaults. For evaluations, both percentage split and cross validation methods were used to compare the results with the original paper. The computational resources were provided by Google through the use of Colab.

3.1 Datasets

Dataset Link: <https://archive.ics.uci.edu/ml/datasets/Wine+Quality>

The dataset includes two files with the physicochemical data red wines and white wines. There are 1599 red wine samples and 4898 white wine samples. The physicochemical data consist of 11 features plus the label: fixed acidity,

volatile acidity, citric acid, residual sugar, chlorides, free sulfur dioxide, total sulfur dioxide, density, pH, sulphates, alcohol, quality. For red wines the labels range from 3 to 8; white wine labels range from 3 to 9. The labels for both wines are normally distributed such that the extreme ends are rare, most of the labels are 5 or 6.

The training-testing split for the percentage split evaluation was 80% training 20% testing. For the cross validation the paper details a 10 fold cross validation thus here we also use a 10 fold cross validation. A stratified k fold validation (scikit-learn) was used so that the labels were more evenly distributed between testing and training sets. Preprocessing involved adding a column for the type of wine for each wines dataset. For wine type classification we merged and shuffled the two wine's dataframes together. For both type and quality classifications the relevant labels were saved into variable(s) then the type and quality columns were removed from the feature dataframe used for training and testing.

3.2 Hyperparameters

To reproduce the paper's results, we implemented the algorithms with their default values for hyper parameters which were suggested by Scikit-learn. Most of the results were adjacent to the original results. working with different parameters in the algorithms, we concluded the following outcomes:

- Scaling data in pre-processing steps was able to improve the accuracy for all the studied algorithms. Although these improvements are negligible for Random Forest, they are considerable for Super Vector Machine.
- Based on our investigations, the number of samples for Random Forest approach can affect the accuracy. the tuned value was 40 which lead to the best performance.
- Obtaining a convenient value for number of neighbors in KNN can results in better performance in some cases. In our case, the proper value for the best result was 500. In creasing this value resulted in lower accuracy for the algorithm.

3.3 Experimental setup and code

All of the models were taken from the scikit-learn library. We began with the default settings for all the models except for the parameters specifically mentioned within the original paper. As previously mentioned a training testing split was done during data preprocessing for the percentage split evaluation method. For cross validation the cross_validate method from scikit-learn was utilized. Following the methods from the original paper we implemented a 10 fold cross validation. For the k-fold data separation we used another scikit-learn package called StratifiedKFold which we also used to shuffle the dataset. For each model both evaluation methods were explored. After the initial results were obtained further experiments with scaler features and PCA features were investigated. During evaluations we also used the consistent metrics that the original paper used in order to compare our results. We used precision, recall, f1, and Receive Operator Curve (area under the curve). For the multiclass quality classification the metrics were measured using the 'weighted' averaging technique. In order to experiment with scaling the feature we used the StandardScaler package. The principal component analysis was performed using scikit-learns PCA package.

3.4 Computational requirements

With the free Google Colaboratory (Colab) we had access to the following computation resource specifications: Operates under Ubuntu 17.10 64 bits and it is composed of an Intel Xeon processor (not specified) with two cores @2.3 GHz and 13 GB RAM. It is equipped with a NVIDIA Tesla K80 (GK210 chipset) 12 GB RAM, 2496 CUDA cores @560 MHz [6].

Here are the average runtime for each experiment:

Percentage-Split (PS) Cross-validation (CV)

Type Classification

Random Forest: PS: 1 sec CV: 9 sec

SVM: PS: 7 sec CV: 1 min 38 sec

KNN: PS: 0.5 sec CV: 1 sec

Red Wine Type Classification

	RF	SVM	KNN
PS	0.5 sec	1 sec	0.7 sec
CV	4 sec	9 sec	0.7 sec

White Wine Type Classification

	RF	SVM	KNN
PS	0.5 sec	14 sec	1 sec
CV	12 sec	2 min 46 sec	1 sec

4 Results

In our wine type classification results we found that cross-validation KNN accuracy reported were higher than the original paper's results. While our results still support that Random Forest algorithm performs the best with an accuracy of 99.8% it was now marginally better than the KNN (99.6%). For percentage split analysis we observed that the SVM actually over performs in accuracy (99.6%) than the Random Forest (99.5%); from percentage split perspective the original paper's claims are disproven.

For the quality classifications the cross validation and percentage split show that Random Forest has the highest accuracy for both red wine and white wine classifications. This is consistent with the original paper's results, supporting their claim that Random Forest is the best performing model for quality classification.

After applying PCA to the features we observed that red wine's cross validation accuracy increased from 70.9% to 72.6% and the percentage split accuracy increased from 69.0% to 70.9%. For the white wine cross validation decreased from 70.1% to 69.9%. However the percentage split accuracy of white wine actually increased from 70.2% to 71.3%. Here, we see that both evaluations support that red wine classification accuracy increases but for white wine, cross validation supports the original paper's claim that PCA decreases the accuracy while percentage split contradicts this claim.

4.1 Results reproducing original paper

For each experiment, say 1) which claim in Section 2 it supports, and 2) if it successfully reproduced the associated experiment in the original paper. For example, an experiment training and evaluating a model on a dataset may support a claim that that model outperforms some baseline. Logically group related results into sections.

4.1.1 Result 1

The table containing all three methods validation metrics for type classification is represented below.

	Classifier	Precision	Recall	F measure	ROC
Cross Validation	SVM	98.9	99.3	99.1	99.4
	KNN	99.6	99.6	99.6	99.5
	RF	99.5	99.8	99.6	99.9
Percentage Split	SVM	98.7	99.6	99.1	99.3
	KNN	97.02	97.02	97.02	96.8
	RF	99.6	99.5	99.5	99.8

Table 1: Performance results of the classification of red and white wine samples

This experiment was done to support the claim that Random Forest is the best wine type classifier. We were able to reproduce the results in the cross validation but not the percentage split evaluation. In the percentage split SVM had a higher accuracy.

4.1.2 Result 2

In this section, the performance of all three classifiers illustrated in Table 2 for quality classification of red and white wines. This experiment was done to support the claim that Random Forest is the best performing classifier of wine quality for both red and white wines. We were able to successfully reproduce the experiment associated with this claim because the accuracy of Random Forest are the highest for both red and white wines.

	Classifier	Precision		Recall		F measure		ROC	
		Red	White	Red	White	Red	White	Red	White
Cross Validation	SVM	56.8	44.7	29.3	28.9	35.9	29.5	75.22	70.1
	KNN	64.3	65.6	64.5	65.8	64.2	65.6	63.5	69.8
	RF	68.1	70.7	70.9	70.1	69.07	69.1	84.8	85.04
Percentage Split	SVM	62.1	47.4	35.6	26.7	40.8	29.1	74.6	69.5
	KNN	54.8	56.7	56.2	57.1	55.4	56.9	65.1	67.9
	RF	66.5	70.3	69.0	70.2	67.7	69.2	84.8	87.5

Table 2: Performance results of the classification of red and white wine sample qualities

4.1.3 Result 3

The result corresponding to the third claim of the paper mentioned in Section 2 where the results are provided after applying the PCA for all classifiers are represented in Table3.

	Classifier	Precision		Recall		F measure		ROC	
		Red	White	Red	White	Red	White	Red	White
Cross Validation	SVM	56.8	44.7	29.3	28.98	35.9	29.5	75.2	69.9
	KNN	64.3	65.6	64.5	65.8	64.2	65.6	63.5	69.8
	RF	69.9	71.4	72.6	69.9	70.4	68.8	85.9	84.5
Percentage Split	SVM	62.1	47.5	35.6	26.7	40.8	29.2	74.5	69.5
	KNN	54.8	56.7	56.2	57.1	55.4	56.9	65.1	67.9
	RF	67.8	72.27	70.9	71.3	68.9	70.4	84.7	87.9

Table 3: Performance results of the classification of red and white wine sample qualities after applying PCA

This experiment was done to support the claim that PCA increases the accuracy quality classification of red wine but decrease the accuracy of white wines. We were able to reproduce the associated experiment in all evaluation methods except the percentage split of the white wine. The percentage split of white wine shows that it increases with PCA features.

4.2 Results beyond original paper

The parameters provided in scikit-learn for each algorithm were examined and the considerable results are as follows:

- Scaling data:

	Classifiers	Precision	Recall	F measure	ROC
without scaling	SVM	56.8	29.3	35.9	75.5
	KNN	63.1	64.7	62.4	72.6
	RF	69.9	72.6	70.4	85.9
with scaling	SVM	53.9	36.02	40.7	80.2
	KNN	67.09	70.04	67.9	80.5
	RF	69.7	72.4	70.2	86.03

Table 4: Scaling impact on performance for quality classification

- Number of neighbors in KNN: According to the results obtained, the performance metric values are presented in Table5.

	number of neighbours	Precision	Recall	F measure	ROC
KNN	1	64.3	64.5	64.2	63.5
	10	65.4	67.9	66.3	75.6
	100	66.9	69.4	67.1	83.9
	500	68.1	68.2	65.0	87.8

Table 5: Number of neighbours impact on KNN performance

5 Discussion

Here we tried to reproduce the experiments from Yesim Er and Ayten Atasoy, "The Classification of White Wine and Red Wine According to Their Physicochemical Qualities" [1]. In our experiments we found that the cross validation evaluations were able to support all the claims and reproduce all the experiments from the original paper. However the percentage split had contradictions with the wine type classification claim and the PCA decreases white wine accuracy claim.

From the experiments and our own analysis we find that cross validation give a more accurate result for comparison. This is because cross validation is able to utilize the entire dataset to train and test for the classification and the results are averaged out. In this way cross validation is less dependent on the random partitioning of the training-test data and the variance of the results are significantly lower than the percentage split evaluation method. Thus it is more fair to judge the results of our experiment by cross validation rather than percent split.

Judging by our cross validation results we can confidently say that our implementation of the experiments are acceptable to recreate the results of the original paper and thus it is able to reestablish the claims of the paper.

In our experimental search beyond that of the paper by Er and Ayten we highlight two findings: SVM and KNN benefit from standardizing the features by removing the mean and scaling to unit variance and KNN's number of neighbors parameter peaks accuracy at 100 and at 500 it begins to decrease so there is an optimal number of neighbors to increase the accuracy of KNN.

One strength of our approach is we implemented all the models through a scikit-learn package. This is very helpful because we had assurance that the models are implemented correctly and that the model implementation themselves are not the source of the discrepancies in our results. Also evaluating with both cross validation and percentage split gave us multiple results to draw an analysis from; if we only use percentage split our conclusions could have been wildly different.

A weaknesses with our approach was that since the authors did not publish their code it was almost impossible to recreate the exact methods they used. So getting the models report the same accuracy for all the experiments was infeasible. The authors also left out much of the parameter details for the models making it extra difficult to even reimplement the method. We also did not have enough time to fine tune the hyperparameters to match the results of the original paper.

References

- [1] Er, Yeşim. 2016. "The Classification of White Wine and Red Wine According to Their Physicochemical Qualities." *International Journal of Intelligent Systems and Applications in Engineering* 4(Special Issue-1):23–26. doi: 10.18201/ijisae.265954.
- [2] scikit-learn: machine learning in Python - mdash; scikit-learn 0.16.1 documentation. (n.d.). Scikit-learn: Machine Learning in Python - Mdash; Scikit-learn 0.16.1 Documentation. <https://scikit-learn.org/>
- [3] P. Cortez, A. Cerderia, F. Almeida, T. Matos, and J. Reis, "Modelling wine preferences by data mining from physicochemical properties," In *Decision Support Systems*, Elsevier, 47 (4): 547-553. ISSN: 0167-9236.
- [4] S. Ebeler, "Linking Flavour Chemistry to Sensory Analysis of Wine," in *Flavor Chemistry, Thirty Years of Progress*, Kluwer Academic Publishers, 1999, pp. 409-422.
- [5] UCI Machine Learning Repository, Wine quality data set, (Online). Available: <https://archive.ics.uci.edu/ml/datasets/Wine+Quality>.
- [6] T. Carneiro, R. V. Medeiros Da Nóbrega, T. Nepomuceno, G. -B. Bian, V. H. C. De Albuquerque and P. P. R. Filho, "Performance Analysis of Google Colaboratory as a Tool for Accelerating Deep Learning Applications," in *IEEE Access*, vol. 6, pp. 61677-61685, 2018, doi: 10.1109/ACCESS.2018.2874767.